

AG35-CET QuecOpen GPIO API 参考手册

LTE Standard 模块系列

版本：1.0.0

日期：2023-08-17

状态：临时文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2023，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2023.

文档历史

修订记录

版本	日期	作者	变更表述
-	2023-08-17	Gabriel LI	文档创建
1.0.0	2023-08-17	Gabriel LI	临时版本

目录

文档历史	3
目录	4
表格索引	6
1 引言	7
2 内核层 GPIO 配置 API	8
2.1. 头文件	8
2.2. 函数概览	8
2.3. 函数详解	9
2.3.1. gpio_request	9
2.3.2. gpio_direction_input	9
2.3.3. gpio_direction_output	9
2.3.4. gpio_get_value	10
2.3.5. gpio_set_value	10
2.3.6. gpio_to_irq	11
2.3.7. gpio_free	11
2.4. 中断 API 函数详解	12
2.4.1. request_irq	12
2.4.2. free_irq	12
2.5. 示例代码	13
2.5.1. pinctrl-names 方式	13
2.5.2. 自定义字段方式	14
2.5.2.1. 普通 GPIO 示例	14
2.5.2.2. GPIO 中断示例:	15
3 用户层 GPIO 配置 API	16
3.1. 头文件	16
3.2. 函数概览	16
3.3. 函数详解	17
3.3.1. ql_gpio_init	17
3.3.1.1. ENUM_PINNAME	17
3.3.1.2. ENUM_PIN_DIRECTION	19
3.3.1.3. ENUM_PIN_LEVEL	19
3.3.1.4. ENUM_PIN_PULLSEL	19
3.3.1.5. 返回值	20
3.3.2. ql_gpio_uninit	20
3.3.3. ql_gpio_base_init	21
3.3.4. ql_gpio_set_level	21
3.3.5. ql_gpio_get_level	22
3.3.6. ql_gpio_set_direction	22
3.3.7. ql_gpio_get_direction	23
3.3.8. ql_gpio_set_pull_selection	23

3.3.9.	ql_gpio_get_pull_selection.....	23
3.3.10.	ql_eint_enable	24
3.3.10.1.	ENUM_EINT_TYPE	25
3.3.11.	ql_eint_disable	25
3.3.12.	ql_set_gpio_function	25
3.3.13.	ql_get_gpio_function	26
3.3.14.	ql_check_pin_function_status.....	26
3.4.	GPIO 配置	27
3.4.1.	GPIO 快速配置	27
3.4.2.	GPIO 单独配置	28
3.5.	编译及功能验证	28
3.5.1.	示例介绍及编译	28
3.5.2.	功能验证	28
4	附录 参考文档及术语缩写	30

表格索引

表 1: 内核层 GPIO 配置 API 函数概览	8
表 2: 用户层 GPIO 配置 API 函数概览	16
表 3: 参考文档	30
表 4: 术语缩写	30

Preliminary Document
Not Checked

1 引言

移远通信 AG35-CET 模块支持 QuecOpen®方案。QuecOpen®是基于 Linux 的嵌入式开发平台，可简化 IoV 应用的软件设计和开发过程。有关 QuecOpen®的详细信息，请参考文档 [1]。

移远通信 AG35-CER QuecOpen®模块支持 GPIO 功能。本文档从用户开发角度，分别介绍了内核层和用户层 GPIO 配置相关的 API 接口函数以及相关示例，帮助实现简易而快速的开发。

2 内核层 GPIO 配置 API

2.1. 头文件

内核层 GPIO API 头文件为 `gpio.h`，位于 `ql-ol-kernel/include/linux/` 目录下。内核层中断 API 头文件为 `interrupt.h`，亦位于 `ql-ol-kernel/include/linux/` 目录下。若无特别说明，本文档所提及的头文件均位于该目录下。

2.2. 函数概览

表 1：内核层 GPIO 配置 API 函数概览

函数	描述
<code>gpio_request()</code>	申请 GPIO
<code>gpio_direction_input()</code>	设置 GPIO 的使用方向为输入
<code>gpio_direction_output()</code>	设置 GPIO 的使用方向为输出以及输出的电平状态
<code>gpio_get_value()</code>	获取 GPIO 引脚的电平状态
<code>gpio_set_value()</code>	设置 GPIO 引脚的电平状态
<code>gpio_to_irq()</code>	将 GPIO 引脚号转换为相应的 IRQ 值，返回中断号
<code>gpio_free()</code>	释放 GPIO
<code>request_irq()</code>	配置中断
<code>free_irq()</code>	注销中断

2.3. 函数详解

2.3.1. gpio_request

该函数用于申请 GPIO。

- 函数原型

```
int gpio_request(unsigned gpio, const char *label)
```

- 参数

gpio:

[In] 平台 BB 芯片 GPIO 引脚号。

label:

[In] 自定义一个标签。

- 返回值

0 函数执行成功

其他值 函数执行失败

2.3.2. gpio_direction_input

该函数用于设置 GPIO 的使用方向为输入。

- 函数原型

```
int gpio_direction_input(unsigned gpio)
```

- 参数

gpio:

[In] 平台 BB 芯片 GPIO 引脚号。

- 返回值

0 函数执行成功

其他值 函数执行失败

2.3.3. gpio_direction_output

该函数用于设置 GPIO 方向为输出以及输出的电平状态。

- 函数原型

```
int gpio_direction_output(unsigned gpio, int value)
```

- 参数

gpio:

[In] 平台 BB 芯片 GPIO 引脚号。

value:

[In] 设置的电平状态。0 表示低电平；1 表示高电平。

- 返回值

0 函数执行成功

其他值 函数执行失败

2.3.4. gpio_get_value

该函数用于获取 GPIO 引脚的电平状态。

- 函数原型

```
int gpio_get_value(unsigned gpio)
```

- 参数

gpio:

[In] 平台 BB 芯片 GPIO 引脚号。

- 返回值

0 低电平

1 高电平

其他值 函数执行失败

2.3.5. gpio_set_value

该函数用于设置 GPIO 引脚的电平状态。

- 函数原型

```
void gpio_set_value(unsigned gpio, int value)
```

- 参数

gpio:

[In] 平台 BB 芯片 GPIO 引脚号。

value:

[In] 设置的电平状态。0 表示低电平；1 表示高电平。

- 返回值

无

2.3.6. gpio_to_irq

该函数用于将 GPIO 引脚号转换为相应的 IRQ 值，返回中断号。

- 函数原型

```
int gpio_to_irq(unsigned gpio)
```

- 参数

gpio:

[In] 平台 BB 芯片 GPIO 引脚号。

- 返回值

IRQ 值（正数值）	函数执行成功
其他值	函数执行失败

2.3.7. gpio_free

该函数用于释放 GPIO。

- 函数原型

```
void gpio_free(unsigned gpio)
```

- 参数

gpio:

[In] 平台 BB 芯片 GPIO 引脚号。

- 返回值

无

2.4. 中断 API 函数详解

2.4.1. request_irq

该函数用于配置中断。

- 函数原型

```
request_irq(    unsigned int irq, irq_handler_t handler, unsigned long flags,
               const char *name, void *dev)
```

- 参数

irq:

[In] 中断号。由函数 *gpio_to_irq()* 产生。

handler:

[In] 中断处理函数。

flags:

[In] 中断触发方式。边沿触发或电平触发。

name:

[In] 中断处理程序定义的一个标签。

dev:

[In] 传入中断处理程序的参数，默认为 NULL。

- 返回值

0 函数执行成功

其他值 函数执行失败

2.4.2. free_irq

该函数用于注销中断。

- 函数原型

```
void free_irq(unsigned int irq, void *dev_id)
```

- 参数

irq:

[In] 中断号。由函数 *gpio_to_irq()* 产生。

dev_id:

[In] 传入中断处理程序的参数，默认为 NULL，和函数 *request_irq()* 中的 *dev* 参数取值相同。

- 返回值

无

2.5. 示例代码

如下为使用 *pinctrl* 子系统建立的示例代码片段，介绍了两种常见的 GPIO 配置方式，用户可结合该代码和其他驱动代码完成开发流程。

2.5.1. *pinctrl-names* 方式

- a. GPIO 引脚配置代码如下，文件路径为 *ql-ol-kernel/arch/arm/boot/dts/ql-asr1803-overlay.dtsi*。

```
temp_gpio_default { //自定义一个 temp_gpio_default 节点
    pinctrl-single,pins = <
        GPIO25 AF0 /* gpio25 */
    >;
    DS_MEDIUM;PULL_UP;EDGE_BOTH;LPM_NONE;
}
```

- b. *pinctrl-names* 使用的代码如下，文件路径为 *ql-ol-kernel/arch/arm/boot/dts/qcom/ql-asr1803-overlay.dtsi*。

```
xxx{
    compatible = "xxx";                /* 匹配用户 driver */
    pinctrl-names = "temp_default";    /* 定义 pinctrl name，驱动中使用
                                        pinctrl_lookup_state()接口解析 */
    pinctrl-0 = <& temp_gpio_default>; /* 选中上面定义的 gpio 配置 */
    status = "ok";                    /* 使能此设备节点 */
}
```

- c. 设备树配置后，在相关引脚使用的驱动中解析以上配置，相关内核 API 如下：

- 1) 获取一个 *pinctrl* 句柄。

```
/**
 * struct devm_pinctrl_get() - Resource managed pinctrl_get()
 * @dev: the device to obtain the handle for
 *
 * If there is a need to explicitly destroy the returned struct pinctrl,
 * devm_pinctrl_put() should be used, rather than plain pinctrl_put().
```

```
*/
struct pinctrl *devm_pinctrl_get(struct device *dev)
```

2) 获取该引脚对应的引脚状态。

```
/**
 * pinctrl_lookup_state() - retrieves a state handle from a pinctrl handle
 * @p: the pinctrl handle to retrieve the state from
 * @name: the state name to retrieve
 */
struct pinctrl_state *pinctrl_lookup_state(struct pinctrl *p, const char *name)
```

3) 设置引脚为某个状态。

```
/**
 * pinctrl_select_state() - select/activate/program a pinctrl state to HW
 * @p: the pinctrl handle for the device that requests configuration
 * @state: the state handle to select/activate/program
 */
int pinctrl_select_state(struct pinctrl *p, struct pinctrl_state *state)
```

2.5.2. 自定义字段方式

2.5.2.1. 普通 GPIO 示例

a. 在对应的设备信息.dtsi 文件中，其设备节点配置如下：

```
xxxx: xxxxx{
    compatible = "xxxxxxx";
    .....
    Temp-gpio= <&gpio xxx 0>;
    status = "ok";
}
```

b. 在驱动代码中，解析.dtsi 中配置的参数，如下：

```
ret = of_get_named_gpio_flags(np, " temp-gpio ", 0, NULL);

if (ret > 0)
{
    gpio_request(ret, "temp-quec,i2c-en-gpio");
    gpio_direction_output(ret, 1);
    msleep(100);
}
```

2.5.2.2. GPIO 中断示例:

- a. 在对应的设备信息.dtsi 文件中，其设备节点配置如下:

```
xxxx: xxxxx{
    compatible = "xxxxxxx";
    .....
    interrupt-parent = <&gpio>;
    interrupts = <xxx IRQ_TYPE_EDGE_RISING>; /* xxx 是对应的 GPIO 号，其中
                                              IRQ_TYPE_EDGE_RISING 为中断方式 */
    ql,temp-gpio = <&gpioxxx IRQ_TYPE_EDGE_RISING >; /*quec,temp-gpio 自定义字段，
                                                         xxx 是对应的 GPIO 号*/
    status = "ok";
}
```

- b. 在驱动代码中，解析.dtsi 中配置的参数，如下:

```
gpio_pin = of_get_named_gpio_flags(np," ql,temp-gpio", 0, NULL);
ret = gpio_direction_input(gpio_pin);
if (ret < 0) {
    printk("request failed\n");
}
ql_irq = gpio_to_irq(gpio_pin);
ret = request_irq(ql_irq, ql_irq_handler,IRQF_TRIGGER_RISING," ql,temp-gpio", NULL);
if (ret < 0)
    printk("request handle failed\n")
```

- c. 实现中断处理函数的代码示例如下:

```
static irqreturn_t ql_irq_handler (int irq, void *handle){
    printk("xxxxxxxxxxxxxx\n");
}
```

备注

中断处理函数中不可有大量事件或耗时操作，否则会导致系统不响应或其他异常情况。如果需要做一些耗时的操作，建议用户自行添加中断下半部处理机制（一般选择工作队列的方式）。

3 用户层 GPIO 配置 API

3.1. 头文件

GPIO API 头文件为 `ql_gpio.h`，位于 `ql-sysroots/usr/include/ql-sdk/` 目录下。若无特别说明，本章节所提到的头文件均在该目录下。

3.2. 函数概览

表 2：用户层 GPIO 配置 API 函数概览

函数	描述
<code>ql_gpio_init()</code>	对指定的 GPIO 进行初始化配置，包括引脚方向、电平、内部上下拉
<code>ql_gpio_uninit()</code>	释放指定的 GPIO 引脚
<code>ql_gpio_base_init()</code>	初始化 GPIO
<code>ql_gpio_set_level()</code>	当 GPIO 的方向为输出时，配置输出的电平
<code>ql_gpio_get_level()</code>	获取当前 GPIO 的电平状态
<code>ql_gpio_set_direction()</code>	配置引脚方向
<code>ql_gpio_get_direction()</code>	获取引脚方向配置
<code>ql_gpio_set_pull_selection()</code>	配置指定引脚的内部上下拉状态
<code>ql_gpio_get_pull_selection()</code>	获取引脚在模块内部的上下拉状态
<code>ql_eint_enable()</code>	使能引脚中断，并注册用户回调函数
<code>ql_eint_disable()</code>	注销引脚中断功能
<code>ql_set_gpio_function()</code>	配置指定 GPIO 引脚的功能
<code>ql_get_gpio_function()</code>	获取指定 GPIO 引脚的功能

`ql_check_pin_function_status()` 检查引脚功能

3.3. 函数详解

3.3.1. ql_gpio_init

该函数用于对指定的 GPIO 进行初始化配置，包括引脚方向、电平、内部上下拉。

● 函数原型

```
int ql_gpio_init(ENUM_PINNAME      pin_name,
                 ENUM_PIN_DIRECTION dir,
                 ENUM_PIN_LEVEL      level,
                 ENUM_PIN_PULLSEL    pull_sel
                 )
```

● 参数

pin_name:

[In] GPIO 引脚号。详见第 3.3.1.1 章。

dir:

[In] 引脚方向。详见第 3.3.1.2 章。

level:

[In] 引脚电平状态。详见第 3.3.1.3 章。

pull_sel:

[In] 内部上下拉。详见第 3.3.1.4 章。

● 返回值

RES_OK

函数执行成功

RES_IO_NOT_SUPPORT

执行函数失败，输入的 GPIO 无效。详见第 3.3.1.5 章。

RES_IO_ERROR

执行函数失败，I/O 错误。详见第 3.3.1.5 章。

3.3.1.1. ENUM_PINNAME

GPIO 引脚号枚举信息定义如下：

```
typedef enum{
    PINNAME_BEGIN = -1,
    PINNAME_GPIO_123 = 123,
```

```
PINNAME_GPIO_117 = 117,
PINNAME_GPIO_19 = 19,
PINNAME_GPIO_118 = 118,
PINNAME_GPIO_120 = 120,
PINNAME_GPIO_23 = 23,
PINNAME_GPIO_126 = 126,
PINNAME_GPIO_119 = 119,
PINNAME_GPIO_34 = 34,
PINNAME_GPIO_36 = 36,
PINNAME_GPIO_35 = 35,
PINNAME_GPIO_33 = 33,
PINNAME_GPIO_49 = 49,
PINNAME_GPIO_50 = 50,
PINNAME_GPIO_24 = 24,
PINNAME_GPIO_21 = 21,
PINNAME_GPIO_22 = 22,
PINNAME_GPIO_48 = 48,
PINNAME_GPIO_55 = 55,
PINNAME_GPIO_56 = 56,
PINNAME_GPIO_57 = 57,
PINNAME_GPIO_59 = 59,
PINNAME_GPIO_58 = 58,
PINNAME_GPIO_43 = 43,
PINNAME_GPIO_32 = 32,
PINNAME_GPIO_31 = 31,
PINNAME_GPIO_52 = 52,
PINNAME_GPIO_51 = 51,
PINNAME_END=127
}ENUM_PINNAME
```

● 成员

成员	描述
<i>PINNAME_GPIO_117</i>	GPIO 引脚编号 117
<i>PINNAME_GPIO_120</i>	GPIO 引脚编号 120
<i>PINNAME_GPIO_19</i>	GPIO 引脚编号 19
<i>PINNAME_GPIO_118</i>	GPIO 引脚编号 118
.....	
<i>PINNAME_GPIO_51</i>	GPIO 引脚编号 51
<i>PINNAME_END</i>	GPIO 最大支持引脚编号 127

3.3.1.2. ENUM_PIN_DIRECTION

引脚方向枚举信息定义如下：

```
typedef enum{
    PINDIRECTION_IN    = 0,
    PINDIRECTION_OUT = 1
}ENUM_PIN_DIRECTION
```

● 成员

成员	描述
<i>PINDIRECTION_IN</i>	输入
<i>PINDIRECTION_OUT</i>	输出

3.3.1.3. ENUM_PIN_LEVEL

引脚电平状态枚举信息定义如下：

```
typedef enum{
    PINLEVEL_LOW = 0,
    PINLEVEL_HIGH = 1
}ENUM_PIN_LEVEL
```

● 成员

成员	描述
<i>PINLEVEL_LOW</i>	低电平
<i>PINLEVEL_HIGH</i>	高电平

3.3.1.4. ENUM_PIN_PULLSEL

引脚上下拉枚举定义如下：

```
typedef enum{
    PINPULLSEL_DISABLE    = 0,
    PINPULLSEL_PULLDOWN  = 1,
    PINPULLSEL_PULLUP     = 2
}ENUM_PIN_PULLSEL
```

- 成员

成员	描述
<i>PINPULLSEL_DISABLE</i>	悬空
<i>PINPULLSEL_PULLDOWN</i>	下拉
<i>PINPULLSEL_PULLUP</i>	上拉

3.3.1.5. 返回值

返回值枚举定义如下：

```
enum {
    RES_OK = 0,
    RES_BAD_PARAMETER = -1,
    RES_IO_NOT_SUPPORT = -2,
    RES_IO_ERROR = -3,
    RES_NOT_IMPLEMENTED = -4
}
```

- 成员

成员	描述
<i>RES_OK</i>	函数执行成功
<i>RES_BAD_PARAMETER</i>	无效参数
<i>RES_IO_NOT_SUPPORT</i>	不支持该 GPIO
<i>RES_IO_ERROR</i>	IO 错误
<i>RES_NOT_IMPLEMENTED</i>	设置不生效

3.3.2. ql_gpio_uninit

该函数用于释放指定的 GPIO 引脚。

- 函数原型

```
int ql_gpio_uninit(ENUM_PINNAME pin_name)
```

- 参数

pin_name:

[In] GPIO 引脚号。详见第 3.3.1.1 章。

- 返回值

RES_OK 函数执行成功

RES_IO_NOT_SUPPORT 执行函数失败，输入的 GPIO 无效。详见第 3.3.1.5 章。

RES_IO_ERROR 执行函数失败，I/O 错误。详见第 3.3.1.5 章。

3.3.3. ql_gpio_base_init

该函数用于初始化 GPIO。根据实际需求，在该函数与 *ql_gpio_init()* 间选取使用。

- 函数原型

```
int ql_gpio_base_init(ENUM_PINNAME pin_name )
```

- 参数

pin_name:

[In] GPIO 引脚号。详见第 3.3.1.1 章。

- 返回值

RES_OK 函数执行成功

RES_IO_NOT_SUPPORT 执行函数失败，输入的 GPIO 无效。详见第 3.3.1.5 章。

RES_IO_ERROR 执行函数失败，I/O 错误。详见第 3.3.1.5 章。

3.3.4. ql_gpio_set_level

该函数用于当 GPIO 的方向为输出时，配置输出的电平。

- 函数原型

```
int ql_gpio_set_level(ENUM_PINNAME pin_name, ENUM_PIN_LEVEL level)
```

- 参数

pin_name:

[In] GPIO 引脚号。详见第 3.3.1.1 章。

level:

[In] 引脚电平状态。详见第 3.3.1.3 章。

- 返回值

<code>RES_OK</code>	函数执行成功
<code>RES_IO_NOT_SUPPORT</code>	执行函数失败，输入的 GPIO 无效。详见第 3.3.1.5 章。
<code>RES_IO_ERROR</code>	执行函数失败，I/O 错误。详见第 3.3.1.5 章。

3.3.5. ql_gpio_get_level

该函数用于获取当前 GPIO 的电平状态。

- 函数原型

```
Int ql_gpio_get_level(ENUM_PINNAME pin_name)
```

- 参数

pin_name:
[In] GPIO 引脚号。详见第 3.3.1.1 章。

- 返回值

<code>0</code>	低电平
<code>1</code>	高电平
<code>RES_IO_NOT_SUPPORT</code>	执行函数失败，不支持该 GPIO。详见第 3.3.1.5 章。

3.3.6. ql_gpio_set_direction

该函数用于配置引脚方向。

- 函数原型

```
int ql_gpio_set_direction(ENUM_PINNAME pin_name, ENUM_PIN_DIRECTION dir)
```

- 参数

pin_name:
[In] GPIO 引脚号。详见第 3.3.1.1 章。

dir:
[In] 引脚方向。详见第 3.3.1.2 章。

- 返回值

<code>RES_OK</code>	函数执行成功
<code>RES_IO_NOT_SUPPORT</code>	执行函数失败，输入的 GPIO 无效。详见第 3.3.1.5 章。
<code>RES_IO_ERROR</code>	执行函数失败，I/O 错误。详见第 3.3.1.5 章。

3.3.7. ql_gpio_get_direction

该函数用于获取引脚方向配置。

- 函数原型

```
int ql_gpio_get_direction(ENUM_PINNAME pin_name)
```

- 参数

pin_name:

[In] GPIO 引脚号。详见第 3.3.1.1 章。

- 返回值

0 输入

1 输出

RES_IO_NOT_SUPPORT 执行函数失败，不支持该 GPIO。详见第 3.3.1.5 章。

3.3.8. ql_gpio_set_pull_selection

该函数用于配置指定引脚的内部上下拉状态。

- 函数原型

```
int ql_gpio_set_pull_selection(ENUM_PINNAME pin_name, ENUM_PIN_PULLSEL pull_sel)
```

- 参数

pin_name:

[In] GPIO 引脚号。详见第 3.3.1.1 章。

pull_sel:

[In] 内部上下拉。详见第 3.3.1.4 章。

- 返回值

RES_OK 函数执行成功

RES_IO_NOT_SUPPORT 执行函数失败，输入的 GPIO 无效。详见第 3.3.1.5 章。

RES_IO_ERROR 执行函数失败，I/O 错误。详见第 3.3.1.5 章。

3.3.9. ql_gpio_get_pull_selection

该函数用于获取引脚在模块内部的上下拉状态。

- 函数原型

```
int ql_gpio_get_pull_selection(ENUM_PINNAME pin_name)
```

- 参数

pin_name:

[In] GPIO 引脚号。详见第 3.3.1.1 章。

- 返回值

0 悬空

1 下拉

2 上拉

RES_IO_NOT_SUPPORT 执行函数失败，不支持该 GPIO。详见第 3.3.1.5 章。

3.3.10. ql_eint_enable

该函数用于使能引脚中断，并注册用户回调函数。

- 函数原型

```
int ql_eint_enable(
    ENUM_PINNAME eint_pin_name,
    ENUM_EINT_TYPE eint_type,
    ql_eint_callback eint_callback
)
```

- 参数

eint_pin_name:

[In] GPIO 引脚号。详见第 3.3.1.1 章。

eint_type:

[In] 边沿触发类型。详见第 3.3.10.1 章。

eint_callback:

[In] 用户回调函数。中断操作会触发该回调函数。

- 返回值

RES_OK 函数执行成功

其他值 执行函数失败。详见第 3.3.1.5 章。

3.3.10.1. ENUM_EINT_TYPE

边沿触发类型枚举定义如下：

```
typedef enum {
    EINT_SENSE_NONE,
    EINT_SENSE_RISING,
    EINT_SENSE_FALLING,
    EINT_SENSE_BOTH
}ENUM_EINT_TYPE
```

- 成员

成员	描述
<i>EINT_SENSE_NONE</i>	引脚无边沿触发中断
<i>EINT_SENSE_RISING</i>	引脚上升沿触发中断
<i>EINT_SENSE_FALLING</i>	引脚下降沿触发中断
<i>EINT_SENSE_BOTH</i>	引脚双边沿触发中断

3.3.11. ql_eint_disable

该函数用于注销引脚中断功能。

- 函数原型

```
int ql_eint_disable(ENUM_PINNAME eint_pin_name)
```

- 参数

eint_pin_name:

[In] GPIO 引脚号。详见第 3.3.1.1 章。

- 返回值

RES_OK

函数执行成功

RES_BAD_PARAMETER

函数执行失败，参数错误。详见第 3.3.1.5 章。

3.3.12. ql_set_gpio_function

该函数用于配置指定 GPIO 引脚的功能。

- 函数原型

```
int ql_set_gpio_function(ENUM_PINNAME pin_name, unsigned int func)
```

- 参数

pin_name:

[In] GPIO 引脚号。详见第 3.3.1.1 章。

func:

[In] GPIO 功能配置，取值范围：0~7。

- 返回值

RES_OK 函数执行成功

其他值 执行函数失败。详见第 3.3.1.5 章。

3.3.13. ql_get_gpio_function

该函数用于获取指定 GPIO 引脚的功能。

- 函数原型

```
int ql_get_gpio_function(ENUM_PINNAME pin_name)
```

- 参数

pin_name:

[In] GPIO 引脚号。详见第 3.3.1.1 章。

- 返回值

0 引脚功能是 GPIO

1 引脚功能不是 GPIO

RES_IO_NOT_SUPPORT 函数执行失败，不支持该 GPIO。详见第 3.3.1.5 章。

RES_IO_ERR 函数执行失败。详见第 3.3.1.5 章。

3.3.14. ql_check_pin_function_status

该函数用于检查引脚功能。

- 函数原型

```
int ql_check_pin_function_status(ENUM_PINNAME pin_name)
```

- 参数

pin_name:

[In] GPIO 引脚号。详见第 3.3.1.1 章。

- 返回值

0

引脚功能是 GPIO

1

引脚功能不是 GPIO

RES_IO_NOT_SUPPORT

函数执行失败，不支持该 GPIO。详见第 3.3.1.5 章。

RES_IO_ERR

函数执行失败。详见第 3.3.1.5 章。

3.4. GPIO 配置

本章节以 GPIO 118 为例介绍如何使用上述 API 对 GPIO 进行配置，包含使用 *ql_gpio_init()* 进行 GPIO 快速配置和使用 *ql_gpio_base_init()* 进行 GPIO 单独配置。

3.4.1. GPIO 快速配置

1. 选择 GPIO 的配置引脚为 GPIO 118，如下图所示：

```
static ENUM_PINNAME m_gpio_pin_gpio = PINNAME_GPIO_118;
```

2. 调用 *ql_check_pin_function_status()* 检查引脚功能，使用方法如下图所示：

```
/*
 *Note:
 * When using gpio configure, you need to determine whether the function of pin is gpio function.
 * If it is not gpio function, the following gpio configure are invalid.
 * You can use the ql_set_gpio_function to setting.
 */
iRet = ql_check_pin_function_status(m_gpio_pin_gpio);
if(iRet == 0){
    iRet = ql_set_gpio_function(m_gpio_pin_gpio,1);
    printf("< Set gpio(%d) function iRet=%d status=%d>\n", m_gpio_pin_gpio,iRet,ql_check_pin_function_status(m_gpio_pin_gpio));
}
```

3. 调用 *ql_gpio_init()* 可以快速对指定的 GPIO 进行初始化配置，使用方法如下图所示：

```
/*
 * Before using gpio init the application layer,you first need to determine whether it is used in the driver.
 */
iRet = ql_gpio_init(m_gpio_pin_gpio, PINDIRECTION_OUT, PINLEVEL_HIGH, PINPULLSEL_DISABLE);
printf("< Init GPIO: pin=%d, dir=%d, level=%d, iRet=%d >\n", m_gpio_pin_gpio, PINDIRECTION_OUT, PINLEVEL_HIGH, iRet);
```

3.4.2. GPIO 单独配置

1. 设置 GPIO 为输入引脚。
调用 `ql_gpio_base_init()` 设置 GPIO 引脚编号为 GPIO 118;
调用 `ql_gpio_set_direction()` 设置引脚输入/输出方向;
调用 `ql_gpio_set_pull_selection()` 设置上下拉。
2. 设置 GPIO 为输出引脚。
调用 `ql_gpio_base_init()` 设置 GPIO 引脚编号为 GPIO 118;
调用 `ql_gpio_set_direction()` 设置引脚输入/输出方向;
调用 `ql_gpio_set_level()` 设置输出电平。
3. 设置 GPIO 普通中断触发。
调用 `ql_gpio_base_init()` 设置 GPIO 引脚编号为 GPIO 118;
调用 `ql_eint_enable()` 设置边沿触发和中断回调函数。

```
iRet = Ql_EINT_Enable(m_gpio_cmpt_pin_gpio, CMPT_EINT_SENSE_BOTH, eint_callback);
printf("< Ql_EINT_Enable: pin=%d iRet=%d >\n", m_gpio_cmpt_pin_gpio, iRet);
```

3.5. 编译及功能验证

3.5.1. 示例介绍及编译

如下示例介绍如何实现每隔 1 秒拉高拉低 GPIO 的功能。进入 `sample/gpio/` 目录，执行 `make` 生成 `gpio_test` 可执行文件，执行 `gpio_test` 即可实现 GPIO 每隔 1 秒拉高拉低的功能。详情如下图所示：

```
ol@ol:~/1803_sec/ql-ol-extsdk-ag35ceraar02a01m2gv01/sample/gpio$
ol@ol:~/1803_sec/ql-ol-extsdk-ag35ceraar02a01m2gv01/sample/gpio$ make
/opt/ql_crosstools/ql-ag35-1803e-gcc-8.4.0-v1-toolchain/bin/arm-openwrt-linux-gcc -o main.o -c main.c -march=armv7-a -marm -
mfpu=neon -mfloat-abi=hard -I. -I/home/ol/1803_sec/ql-ol-extsdk-ag35ceraar02a01m2gv01/ql-sysroots/usr/include -I/home/ol/180
3_sec/ql-ol-extsdk-ag35ceraar02a01m2gv01/ql-sysroots/usr/include/ql-sdk -Wall -Wundef
/opt/ql_crosstools/ql-ag35-1803e-gcc-8.4.0-v1-toolchain/bin/arm-openwrt-linux-gcc -o gpio_test main.o -march=armv7-a -marm -
mfpu=neon -mfloat-abi=hard -L. -L/home/ol/1803_sec/ql-ol-extsdk-ag35ceraar02a01m2gv01/ql-sysroots/usr -L/home/ol/1803_sec/ql
-ol-extsdk-ag35ceraar02a01m2gv01/ql-sysroots/usr/lib -lql_sdk -lpthread -lql_sys_log -lql_lib_ipc -lql_lib_utils
ol@ol:~/1803_sec/ql-ol-extsdk-ag35ceraar02a01m2gv01/sample/gpio$
```

3.5.2. 功能验证

- a. 编译并上传 `gpio_test` 到模块，执行以下命令：

```
adb push <gpio_test 在上位机路径> <模块内部路径，如/mnt>
```

- b. 修改执行权限，执行以下命令：

```
chmod 777 gpio_test
```

- c. 执行 *gpio_test*, 可以看到指定引脚的高低电平变化, 还可以通过万用表测量, 如下:

```

root@OpenWrt:/mnt# ./gpio_test
< OpenLinux: GPIO example >
< Set gpio(118) function iRet=0 status=1>
< Init GPIO: pin=118, dir=1, level=1, iRet=0 >
< Pull pin level to low >
< Pull pin level to high >
< Pull pin level to low >
[ 70.934112] KERNEL-TEXT-CRC: 0x25a0
< Pull pin level to high >
< Pull pin level to low >
< Pull pin level to high >
< Pull pin level to low >
< Pull pin level to high >
< Pull pin level to low >

```

4 附录 参考文档及术语缩写

表 3: 参考文档

文档名称

[1] Quectel_AG35-CET_QuecOpen_快速开发指导

表 4: 术语缩写

缩写	英文描述	中文描述
API	Application Programming Interface	应用程序接口
BB	Baseband	基带
GPIO	General-Purpose Input/Output	通用输入/输出
IoV	Internet of Vehicles	车联网
IRQ	Interrupt Request	中断请求
SDK	Software Development Kit	软件开发工具包